



Digital Design

Chapter 4: Datapath Components

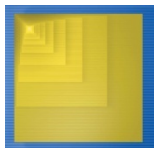
Slides to accompany the textbook *Digital Design*, First Edition,
by Frank Vahid, John Wiley and Sons Publishers, 2007.
<http://www.ddvahid.com>

Copyright © 2007 Frank Vahid

Instructors of courses requiring Vahid's *Digital Design* textbook (published by John Wiley and Sons) have permission to modify and use these slides for customary course-related activities, subject to keeping this copyright notice in place and unmodified. These slides may be posted as unanimated pdf versions on publicly-accessible course websites.. PowerPoint source (or pdf with animations) may not be posted to publicly-accessible websites, but may be posted for students on internal protected sites or distributed directly to students by other electronic means. Instructors may make printouts of the slides available to students for a reasonable photocopying charge, without incurring royalties. Any other use requires explicit permission. Instructors may obtain PowerPoint source or obtain special use permissions from Wiley – see <http://www.ddvahid.com> for information.

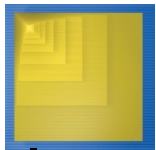
Introduction

- Chapters 2 & 3: Introduced increasingly complex digital building blocks
 - Gates, multiplexors, decoders, basic registers, and controllers
- Controllers good for systems with control inputs/outputs
 - **Control** input: Single bit (or just a few), representing environment event or state, used to influence state of a controller's current state
 - e.g., 1 bit representing button pressed, request for memory transfer, car key signal
 - TV remote (channel selection), calculator (addition)
 - **Data** input: Multiple bits collectively representing single entity
 - e.g., 7 bits representing temperature in binary, memory address, addends
 - TV remote (video signal), calculator (data to be added)
- Need building blocks for data
 - **Datapath components**, aka register-transfer-level (RTL) components, store/transform data
 - Put datapath components together (from simpler components) to form a **datapath**



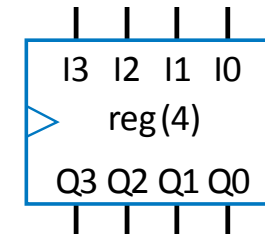
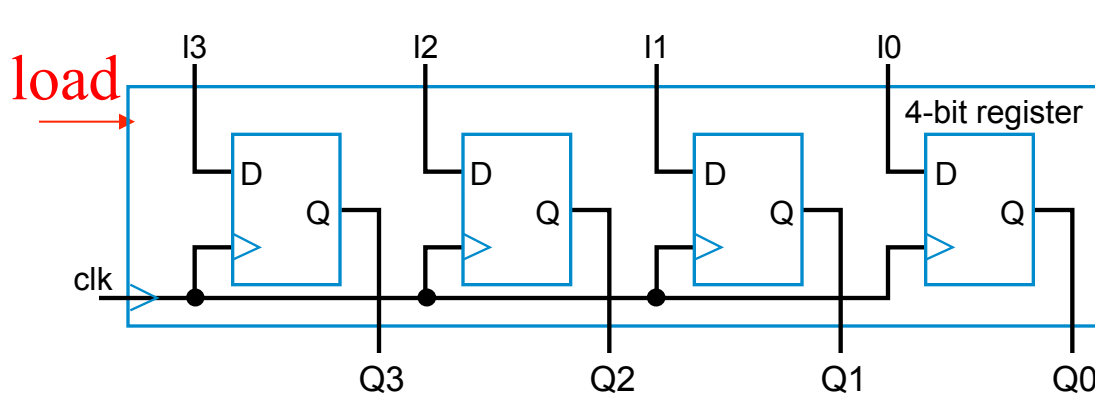
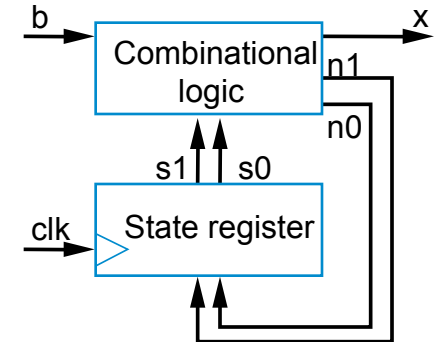
Datapath and Control

- Computer = datapath + control unit + memory
- A **datapath** is a collection of functional units, such as ALUs, adders or multipliers, that perform operations on data.
- Most processors consist of a datapath and a control unit, with a large part of the control unit dedicated to regulating the interaction between the datapath and memory.
- The control unit can be thought of as the brain of the CPU itself. It controls based on the instructions it decodes, how other parts of the CPU and in turn, rest of the computer systems should work in order that the instruction gets executed in a correct manner.
- This chapter introduces numerous datapath components, and simple datapaths
 - Next chapter will combine controllers and datapaths into “processors”



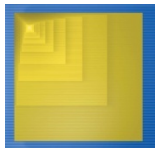
Registers

- Sequential component, Can store data, very common in datapaths
- Basic register of Ch 3: Loaded (stored, written) every cycle
 - Useful for implementing FSM -- stores encoded state
 - For other uses, may want to load only on certain cycles
- Reading (opposite action of loading) does not remove or change bits in the register



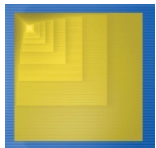
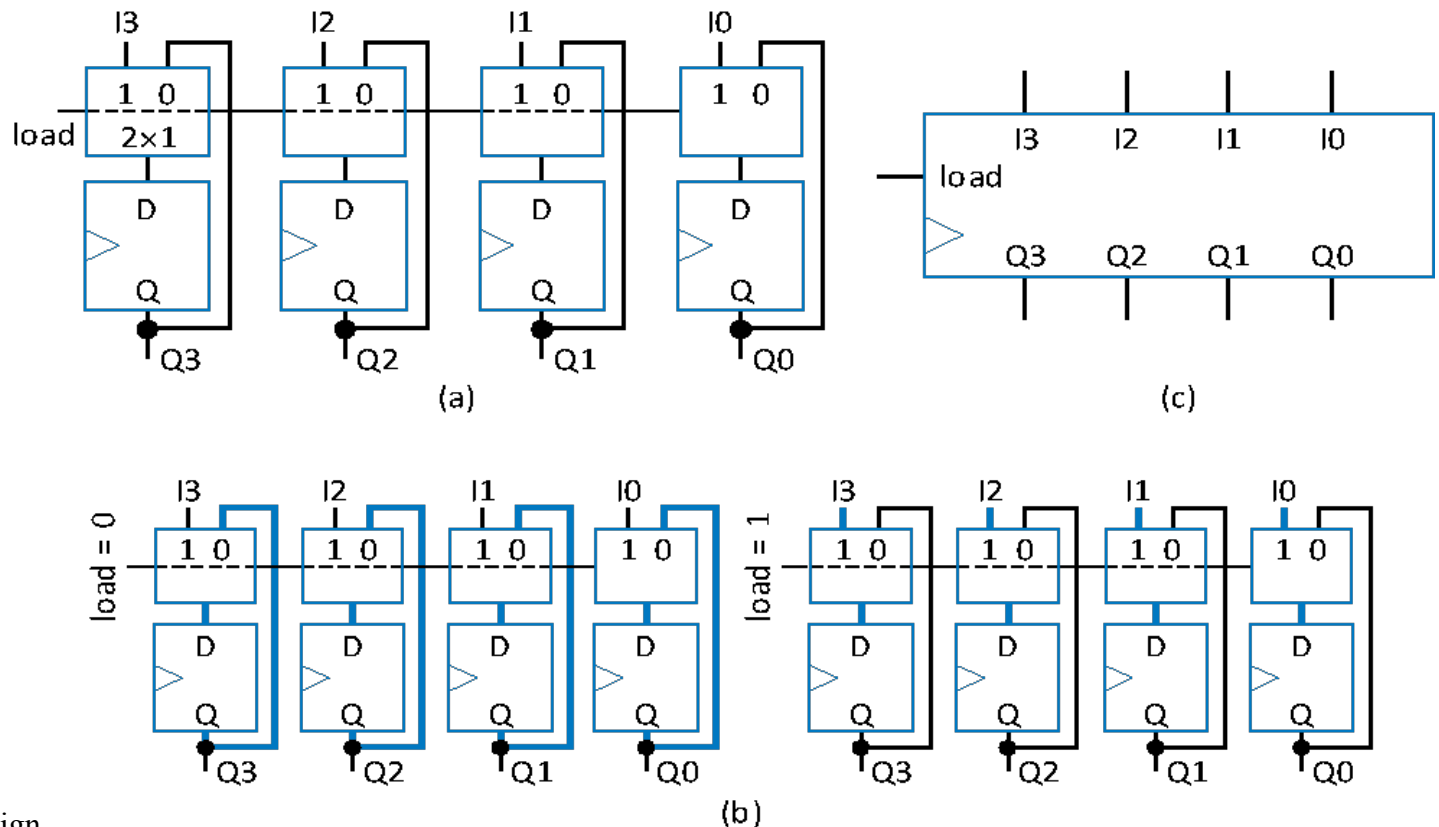
Basic register loads on every clock cycle

How extend to only load on certain cycles?



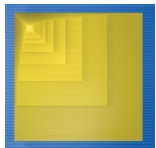
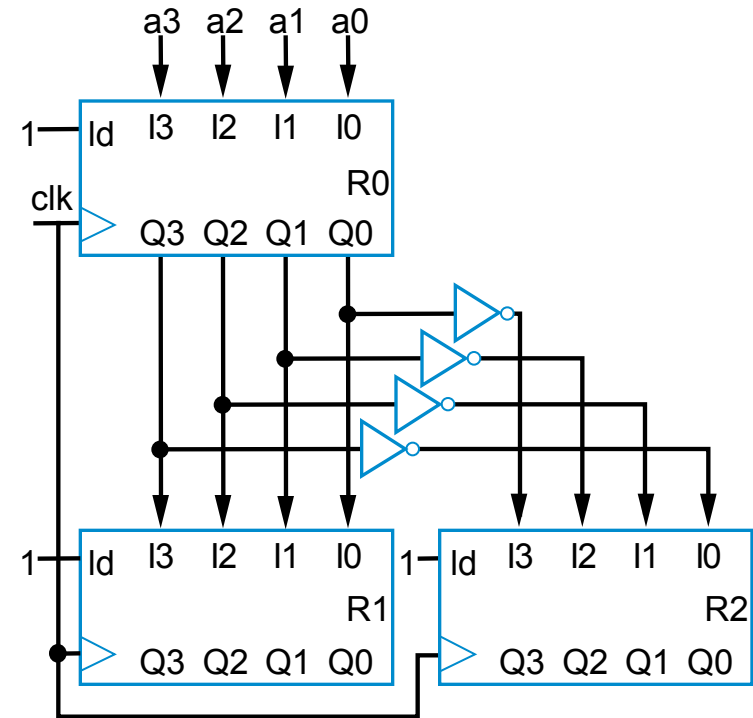
Register with Parallel Load

- Add 2x1 mux to front of each flip-flop
- Register's load input selects mux input to pass
 - Either existing flip-flop value, or new value to load

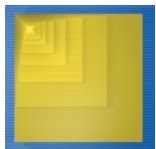
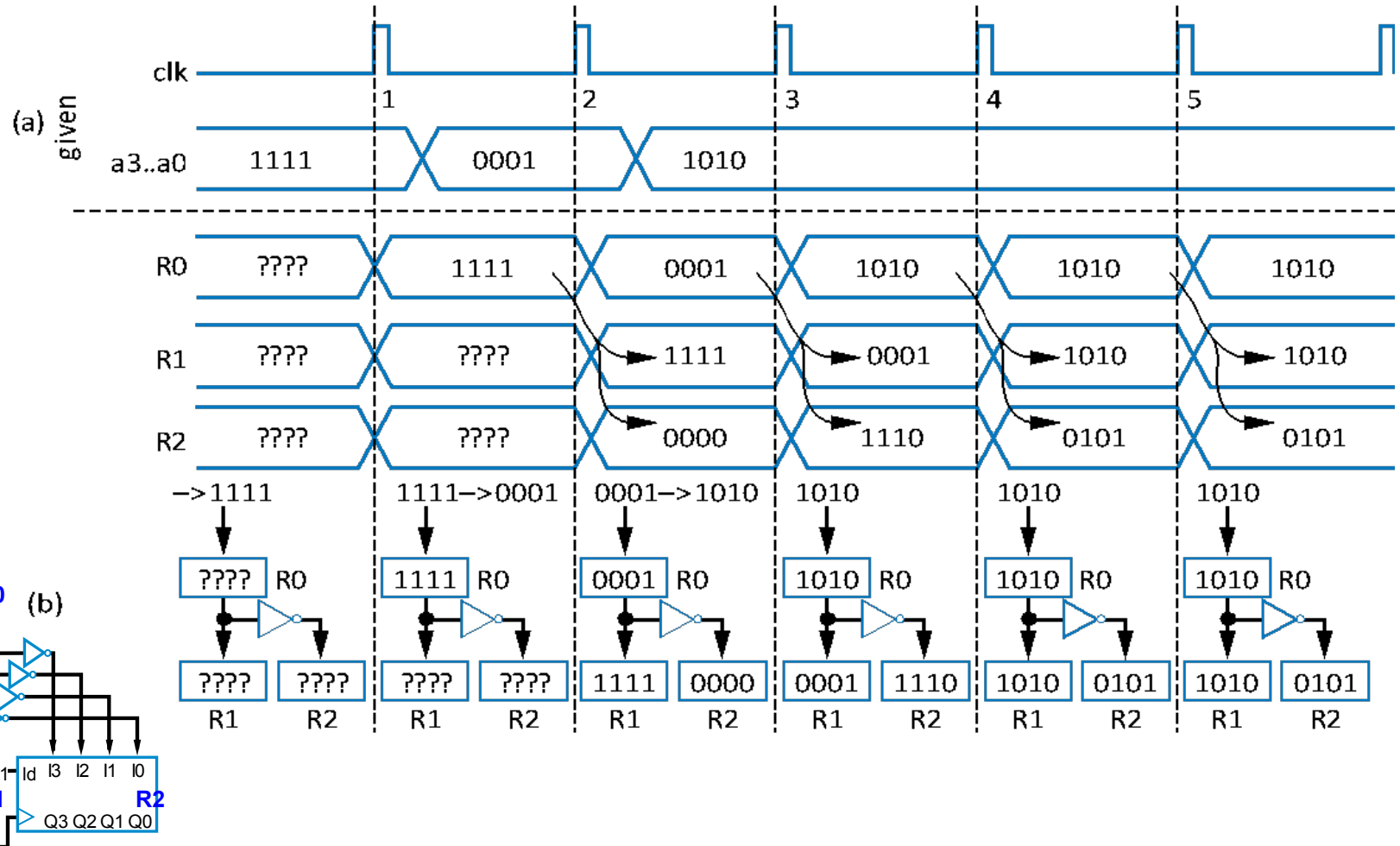


Basic Example Using Registers

- This example will show how registers load simultaneously on clock cycles
 - Notice that all load inputs set to 1 in this example -- just for demonstration purposes

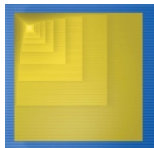
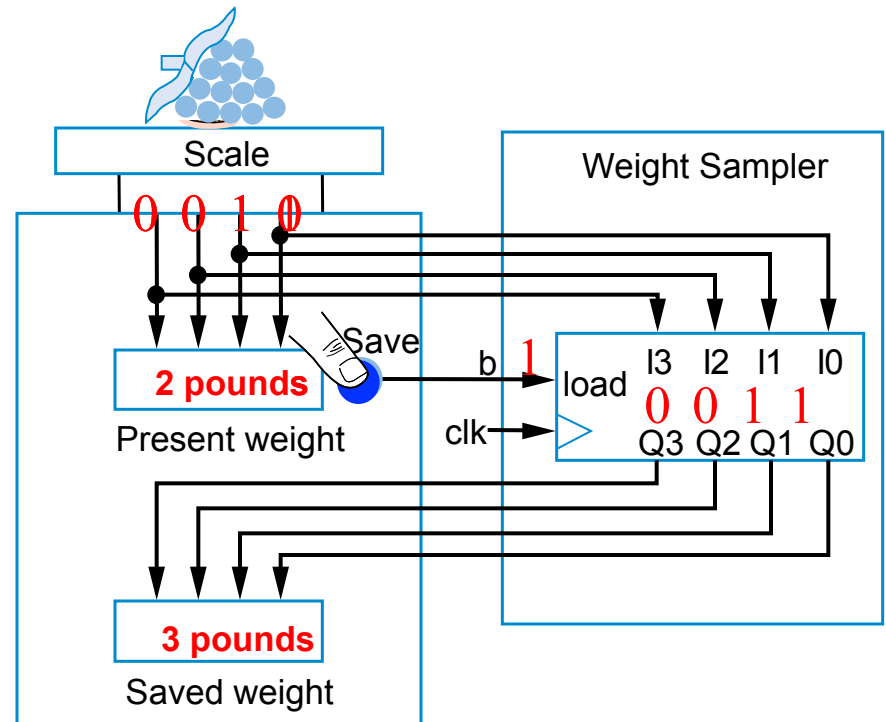


Basic Example Using Registers



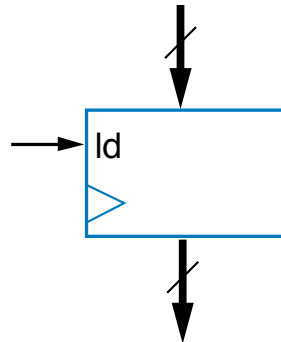
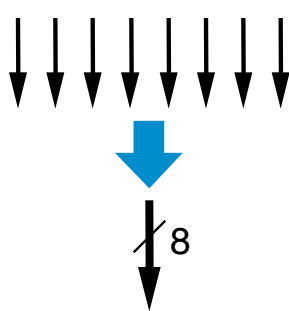
Register Example using the Load Input: Weight Sampler

- Scale has two displays
 - Present weight
 - Saved weight
 - Useful to compare present item with previous item
- Use register to store weight
 - Pressing button causes present weight to be stored in register
 - Register contents always displayed as “Saved weight,” even when new present weight appears

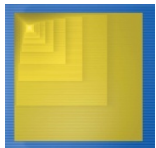
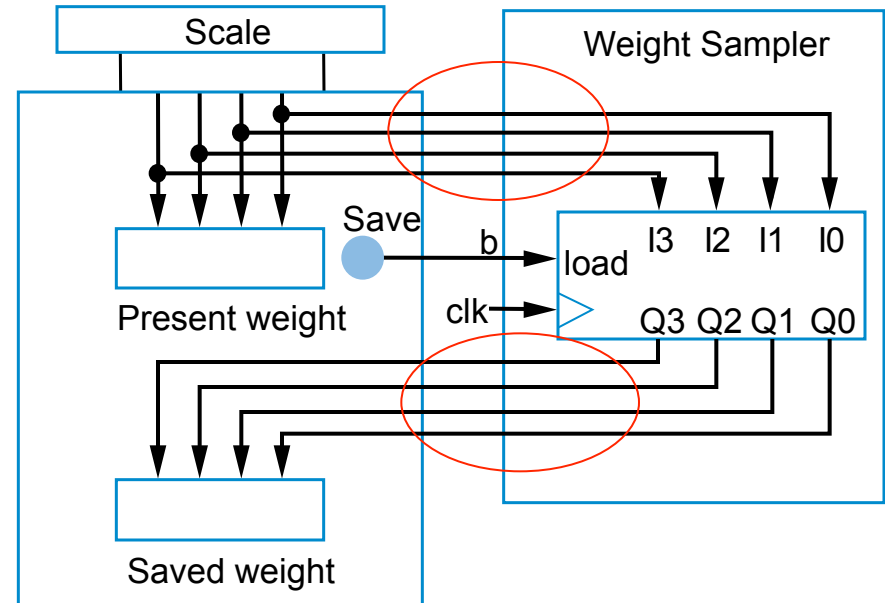


Buses

- N-bit *bus*: N wires to carry N-bit data item
 - Circuit drawings can become cluttered
- Convention for drawing buses
 - Single bold line and/or small angled line across

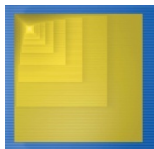
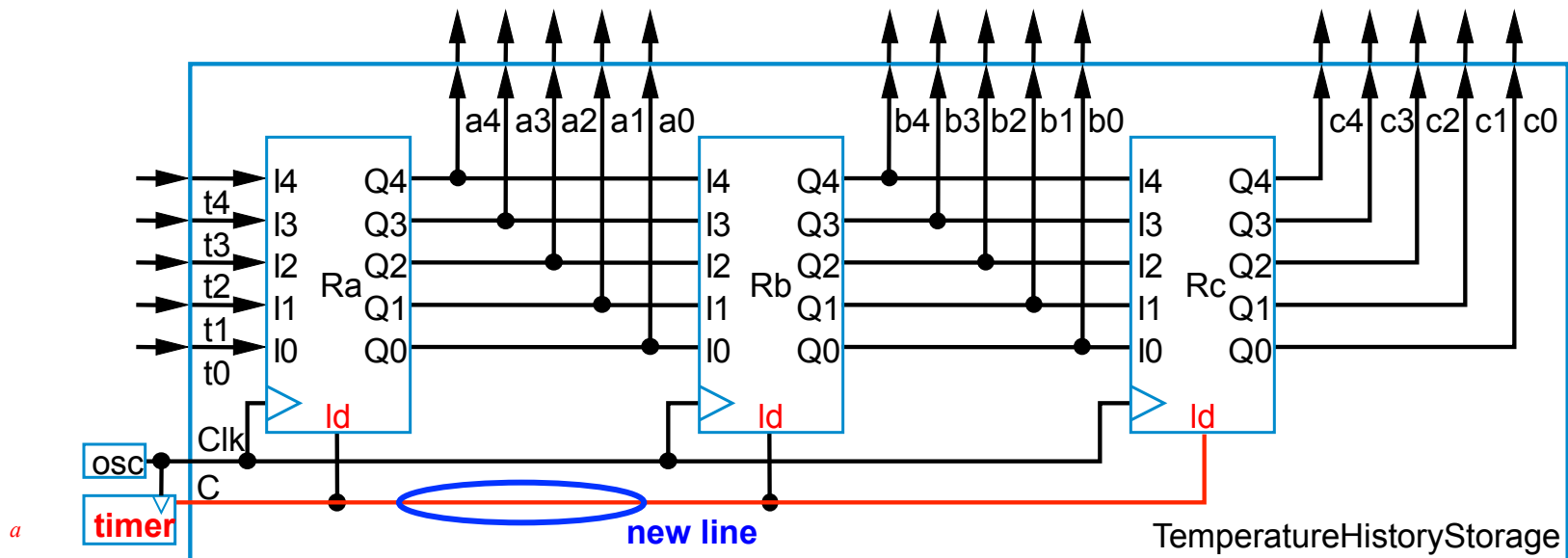


a

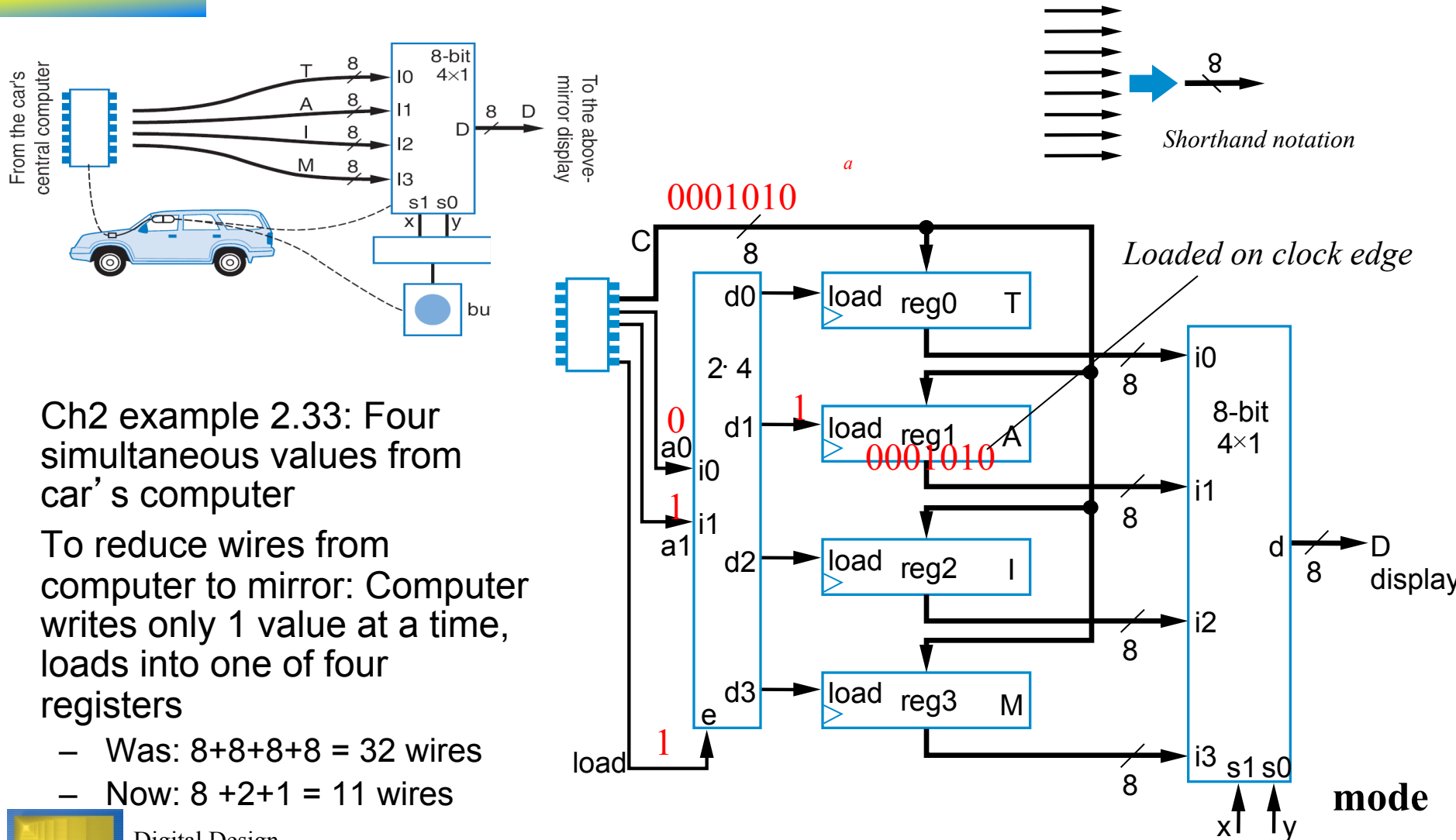


Register Example: Temperature History Display

- Recall Chpt 3 example
 - Timer pulse every hour
 - Previously used as clock. Better design only connects oscillator to clock inputs -- use registers with load input, connect to timer pulse.



Register Example: Above-Mirror Display

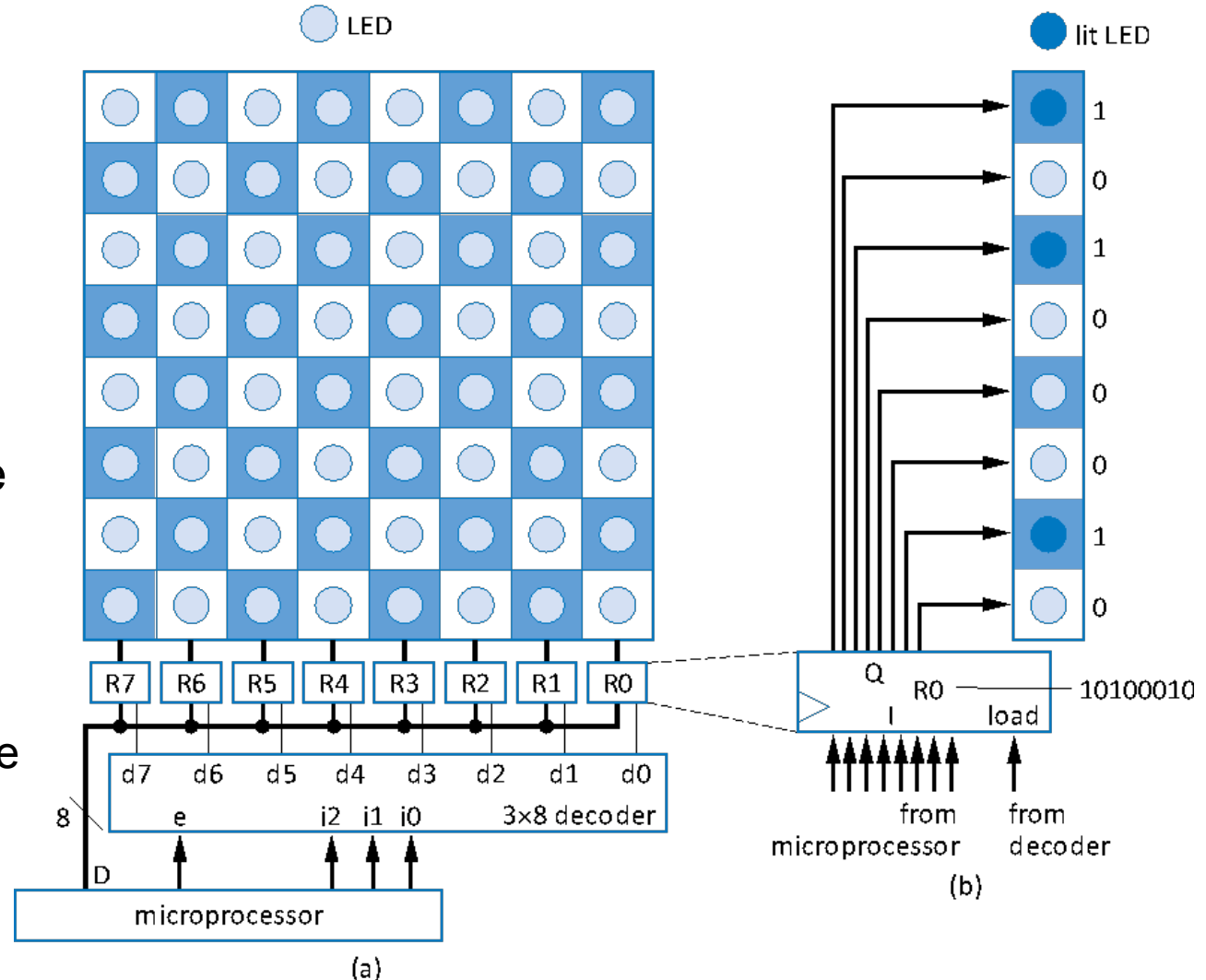


- Ch2 example 2.33: Four simultaneous values from car's computer
- To reduce wires from computer to mirror: Computer writes only 1 value at a time, loads into one of four registers
 - Was: $8+8+8+8 = 32$ wires
 - Now: $8 + 2 + 1 = 11$ wires

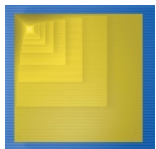
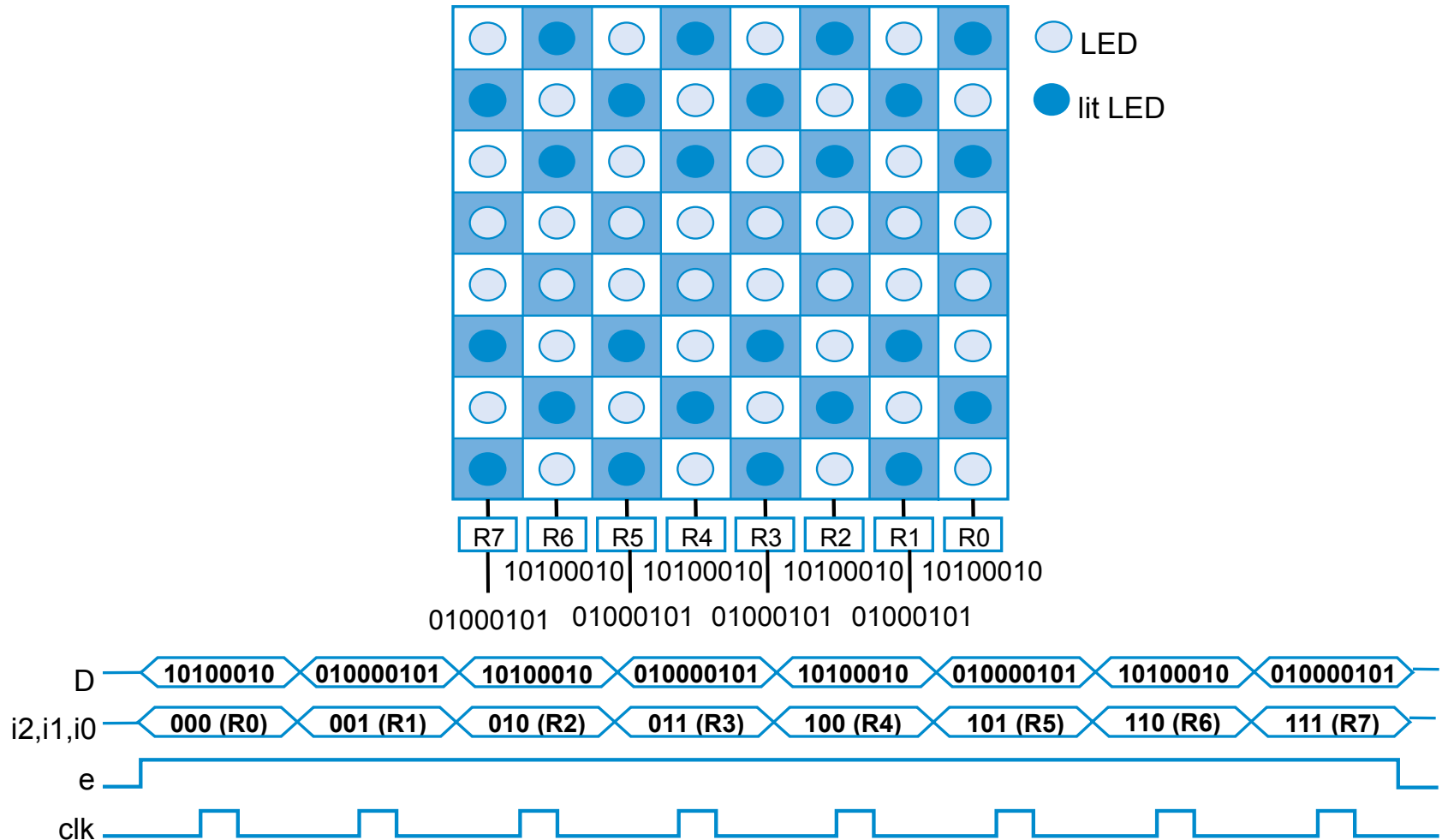


Register Example: Computerized Checkerboard

- Each register holds values for one column of lights
 - 1 lights light
- Microprocessor loads one register at a time
 - Occurs fast enough that user sees entire board change at once

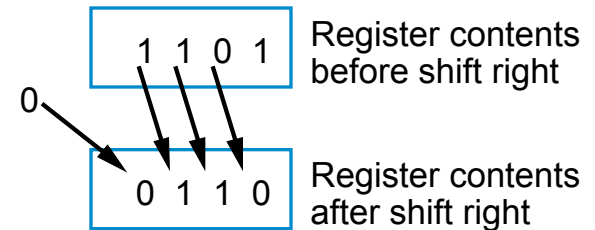


Register Example: Computerized Checkerboard



Shift Register

- Shift right
 - Move each bit one position right
 - Shift in 0 to leftmost bit



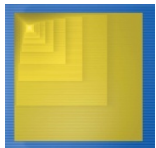
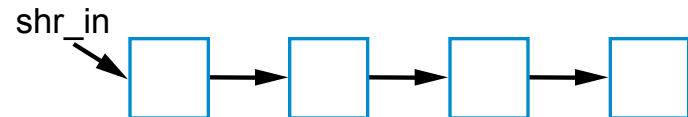
Q: Do four right shifts on 1001, showing value after each shift

A:

1001	(original)
0100	
0010	
0001	
0000	

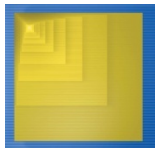
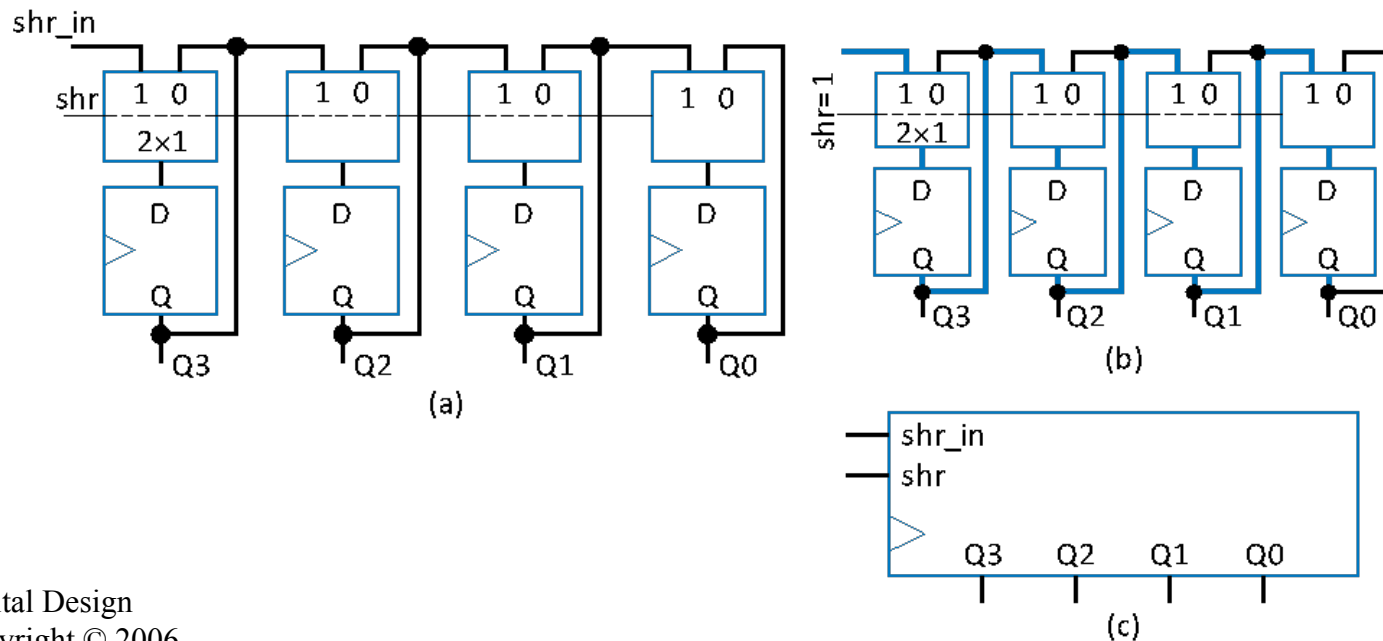
A dashed arrow points from the original value 1001 down to 0001, and a solid arrow points from 0001 down to 0000.

- Implementation: Connect flip-flop output to next flip-flop's input



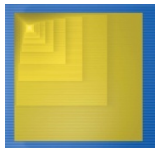
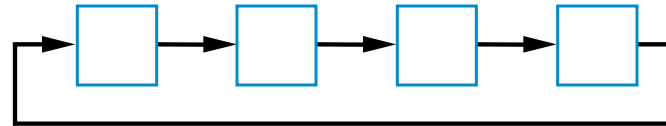
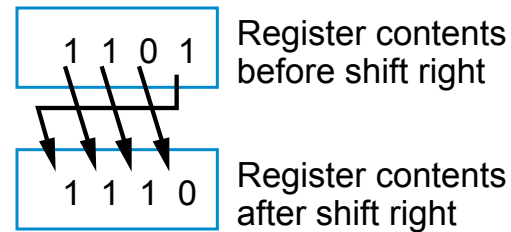
Shift Register

- To allow register to either shift or retain, use 2x1 muxes
 - shr: 0 means retain, 1 shift
 - shr_in: value to shift in
 - May be 0, or 1
- Note: Can easily design shift register that shifts left instead



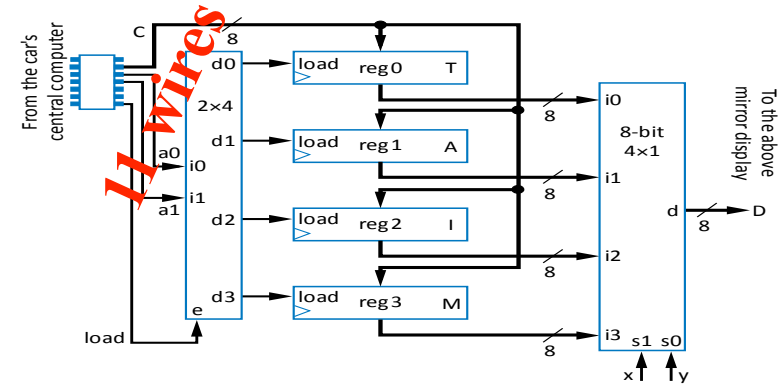
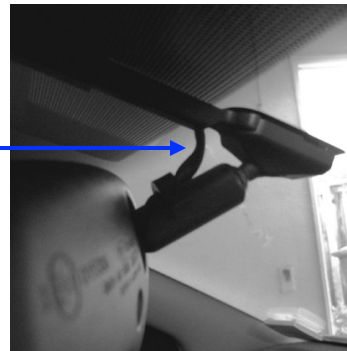
Rotate Register

- Rotate right: Like shift right, but leftmost bit comes from rightmost bit

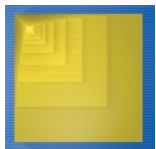
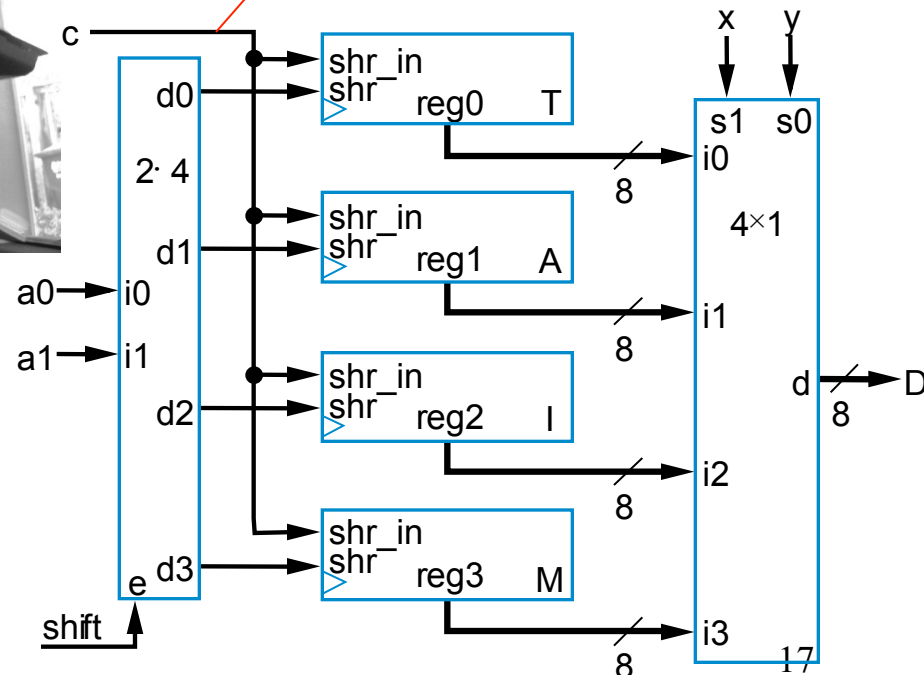


Shift Register Example: Above-Mirror Display

- Earlier example: $8 + 2 + 1 = 11$ wires from car's computer to above-mirror display's four registers
 - Better than 32 wires, but 11 still a lot -- want fewer for smaller wire bundles
- Use shift registers
 - Wires: $1 + 2 + 1 = 4$
 - Computer sends one value at a time, one bit per clock cycle

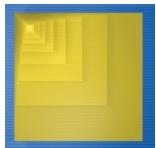


Note: this line is 1 bit, rather than 8 bits like before



Serial Communication

- In serial communication, circuits communicate data by sending the data one bit at a time.
- A serial communication scheme used in automobiles is called **CAN** (Controller Area Network).
- USB, WiFi, Bluetooth are also serial communications. Shift registers are used to convert parallel to serial and vice versa.

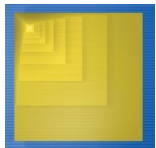
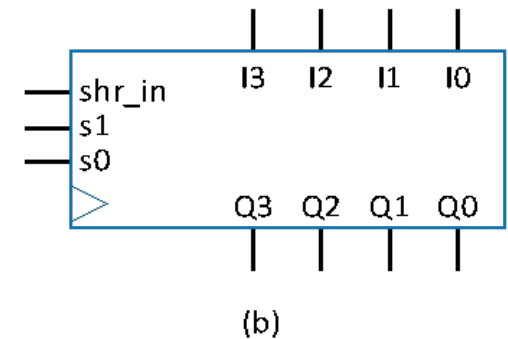
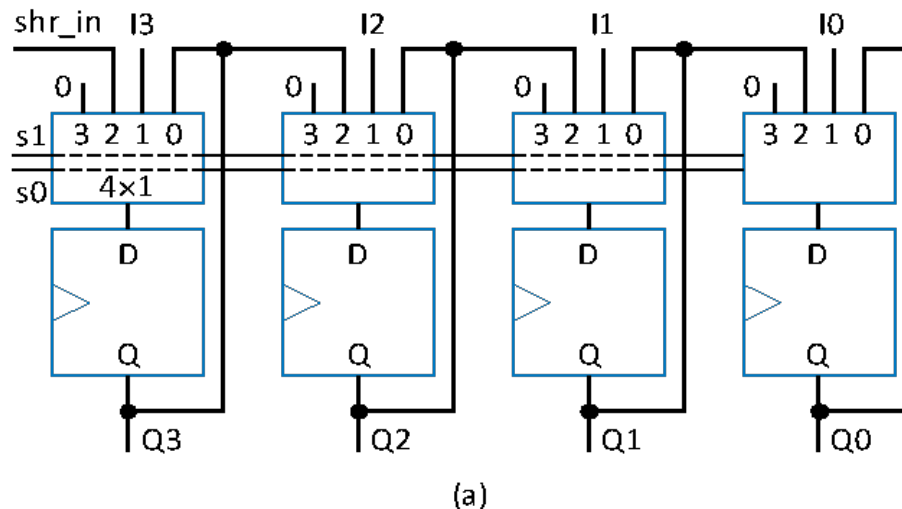


Multifunction Registers (Parallel Load and Shift Right)

- Many registers have multiple functions
 - Load, shift, clear (load all 0s)
 - And retain present value, of course
- Easily designed using muxes
 - Just connect each mux input to achieve desired function

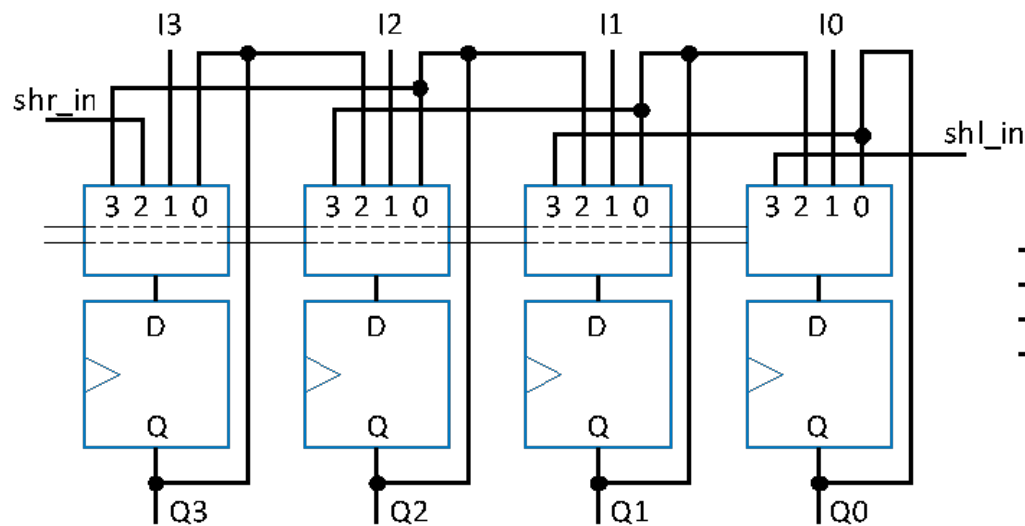
Functions:

s1	s0	Operation
0	0	Maintain present value
0	1	Parallel load
1	0	Shift right
1	1	(unused - let's load 0s)

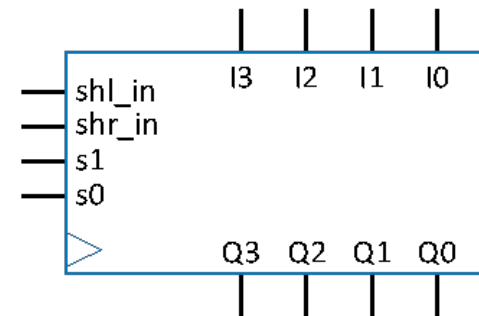


Multifunction Registers (Parallel Load, Shift Left and Shift Right)

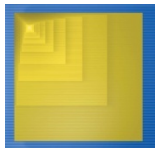
s1	s0	Operation
0	0	Maintain present value
0	1	Parallel load
1	0	Shift right
1	1	Shift left



(a)



(b)

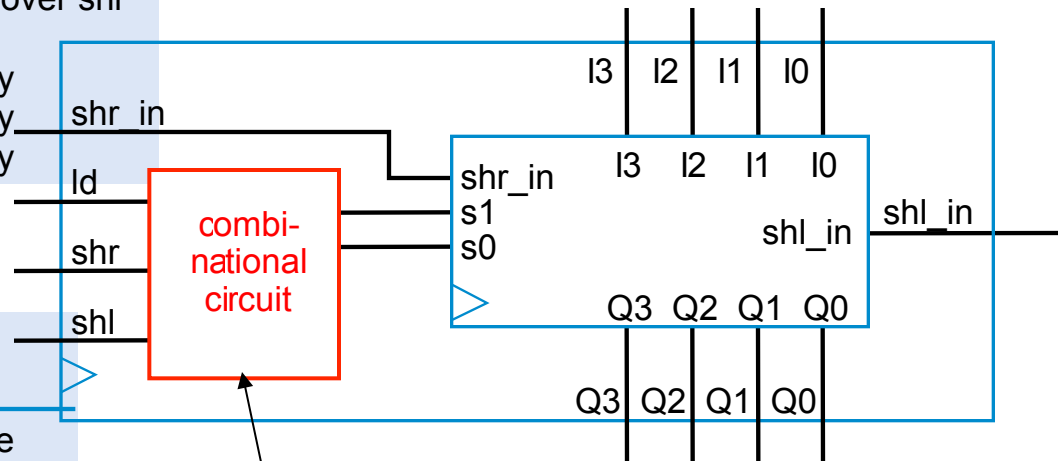


Multifunction Registers with Separate Control Inputs

ld	shr	shl	Operation
0	0	0	Maintain present value
0	0	1	Shift left
0	1	0	Shift right
0	1	1	Shift right – shr has priority over shl
1	0	0	Parallel load
1	0	1	Parallel load – ld has priority
1	1	0	Parallel load – ld has priority
1	1	1	Parallel load – ld has priority

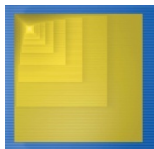
Truth table for combinational circuit

Inputs			Outputs		Note Operation
ld	shr	shl	s1	s0	
0	0	0	0	0	Maintain value
0	0	1	1	1	Shift left
0	1	0	1	0	Shift right
0	1	1	1	0	Shift right
1	0	0	0	1	Parallel load
1	0	1	0	1	Parallel load
1	1	0	0	1	Parallel load
1	1	1	0	1	Parallel load



$$s1 = ld' * shr' * shl + ld' * shr * shl' + ld' * shr * shl$$

$$s0 = ld' * shr' * shl + ld$$

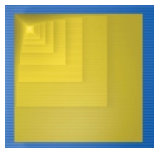


Register Operation Table

- Register operations typically shown using compact version of table
 - X means same operation whether value is 0 or 1
 - One X expands to two rows
 - Two Xs expand to four rows
 - Put highest priority control input on left to make reduced table simple

Inputs			Outputs		Note Operation
ld	shr	shl	s1	s0	
0	0	0	0	0	Maintain value
0	0	1	1	1	Shift left
0	1	0	1	0	Shift right
0	1	1	1	0	Shift right
1	0	0	0	1	Parallel load
1	0	1	0	1	Parallel load
1	1	0	0	1	Parallel load
1	1	1	0	1	Parallel load

ld	shr	shl	Operation
ld	shr	shl	
0	0	0	Maintain value
0	0	1	Shift left
0	1	X	Shift right
1	X	X	Parallel load

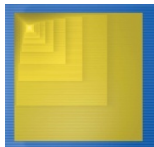


Register Design Process

- Can design register with desired operations using simple four-step process

TABLE 4.1 Four-step process for designing a multifunction register.

	Step	Description
1.	<i>Determine mux size</i>	Count the number of operations (don't forget the maintain present value operation!) and add in front of each flip-flop a mux with at least that number of inputs.
2.	<i>Create mux operation table</i>	Create an operation table defining the desired operation for each possible value of the mux select lines.
3.	<i>Connect mux inputs</i>	For each operation, connect the corresponding mux data input to the appropriate external input or flip-flop output (possibly passing through some logic) to achieve the desired operation.
4.	<i>Map control lines</i>	Create a truth table that maps external control lines to the internal mux select lines, with appropriate priorities, and then design the logic to achieve that mapping



Register Design Example

- Desired register operations
 - Load, shift left, synchronous clear, synchronous set

Step 1: Determine mux size

5 operations: above, plus maintain present value (don't forget this one!)

--> Use 8x1 mux

Step 2: Create mux operation table

Step 3: Connect mux inputs

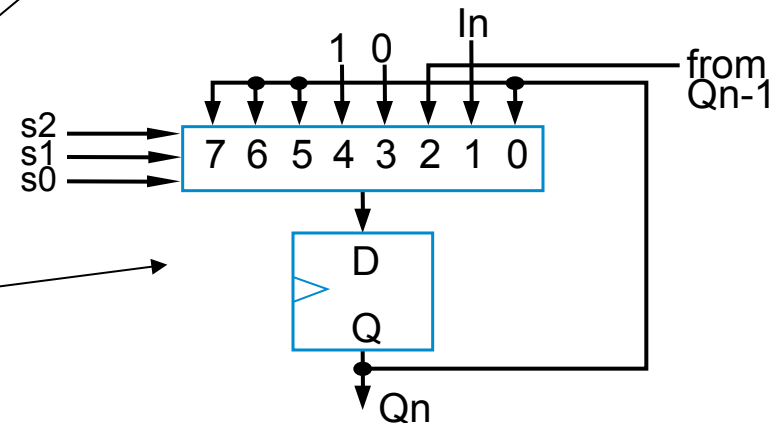
Step 4: Map control lines

$s2 = \text{clr}' * \text{set}$

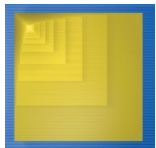
$s1 = \text{clr}' * \text{set}' * \text{ld}' * \text{shl} + \text{clr}$

$s0 = \text{clr}' * \text{set}' * \text{ld} + \text{clr}$

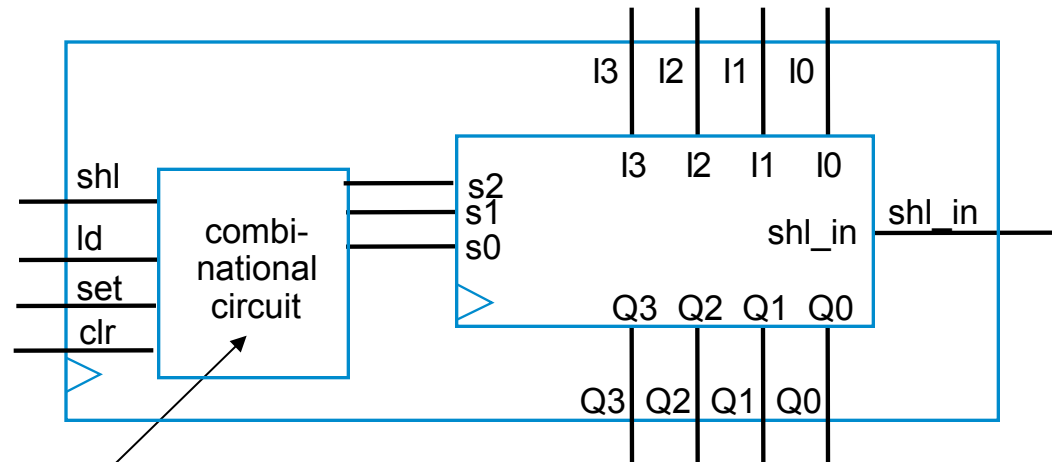
s2	s1	s0	Operation
0	0	0	Maintain present value
0	0	1	Parallel load
0	1	0	Shift left
0	1	1	Synchronous clear
1	0	0	Synchronous set
1	0	1	Maintain present value
1	1	0	Maintain present value
1	1	1	Maintain present value



Inputs				Outputs			Operation
clr	set	ld	shl	s2	s1	s0	
0	0	0	0	0	0	0	Maintain present value
0	0	0	1	0	1	0	Shift left
0	0	1	X	0	0	1	Parallel load
0	1	X	X	1	0	0	Set to all 1s
1	X	X	X	0	1	1	Clear to all 0s



Register Design Example



Step 4: Map control lines

$$s2 = \text{clr}' * \text{set}$$

$$s1 = \text{clr}' * \text{set}' * \text{ld}' * \text{shl} + \text{clr}$$

$$s0 = \text{clr}' * \text{set}' * \text{ld} + \text{clr}$$

Inputs				Outputs			Operation
clr	set	ld	shl	s2	s1	s0	
0	0	0	0	0	0	0	Maintain present value
0	0	0	1	0	1	0	Shift left
0	0	1	X	0	0	1	Parallel load
0	1	X	X	1	0	0	Set to all 1s
1	X	X	X	0	1	1	Clear to all 0s

